

Ejercicio de CoT
Resumen técnico de la solución

Jonathan Ricardo Serna Galina

20074902

Resumen técnico y validación

Documento complementario de BookingCalendar. Presenta una explicación de la solución, decisiones de diseño y evidencia comprobable. No reproduce el razonamiento interno privado del modelo ni se presenta como una transcripción de Chain of Thought.

1. Requisitos identificados

Crear una clase para reservar una sala en un mismo día, rechazar traslapes por sala, cancelar por coincidencia exacta, mostrar disponibilidad en tabla, utilizar formato de 24 horas y seguir PEP 8. El archivo del docente fija los nombres BookingCalendar, book, cancel y get_available_slots.

2. Decisiones de diseño

Representación: diccionario por sala y pares de minutos ordenados. Esta estructura mantiene independientes las agendas y evita depender de paquetes externos.

Intervalos: [inicio, fin). El fin queda disponible para el comienzo de otra reunión. Así, 09:00–11:00 y 11:00–12:00 no entran en conflicto.

Entradas: enteros o cadenas HH:MM. Se rechazan booleanos, decimales, nombres vacíos y horas fuera de rango. 24:00 es únicamente el límite final del día.

Errores: book y cancel devuelven False ante entradas inválidas. La consulta con un nombre inválido produce ValueError para indicar una solicitud mal formada.

3. Ejemplo verificable

Para Sala 1 se registran 08:00–10:00 y 10:00–12:00. Ambas operaciones devuelven True. El intento 09:00–11:00 devuelve False. Antes de agregar otras reservas, la disponibilidad es 00:00–08:00 y 12:00–24:00. Los resultados pueden verificarse directamente mediante los tests incluidos.

4. Cancelación y disponibilidad

La cancelación busca el par exacto de horas dentro de la sala indicada. Una cancelación parcial no elimina una reunión. Después de eliminar la última reserva, la sala permanece registrada y su disponibilidad ocupa el día completo.

La consulta recorre los intervalos ordenados y genera el complemento dentro de 00:00–24:00. Una sala reservada todo el día produce cero huecos. Una sala desconocida consultada explícitamente aparece libre sin modificar el registro.

5. Validación ejecutada

El script del docente se ejecutó correctamente y se conservó sin cambios. Las 13 pruebas adicionales finalizaron con OK. Se comprobaron adyacencia, traslapes, salas independientes, horas y nombres inválidos, minutos, cancelación exacta, huecos ordenados, día completo, sala desconocida, normalización de nombres y salida de tabla.

Las dos expresiones sin assert del archivo original no verifican los horarios imposibles. Los tests adicionales resuelven esa falta de cobertura comprobando que esas llamadas devuelven False.

La revisión pycodestyle de los tres archivos nuevos no reportó infracciones. Las salidas reales se conservan en evidencia_demo.txt, evidencia_docente.txt y evidencia_tests.txt.

6. Complejidad y límites

Para una sala con n reservas, la búsqueda de conflictos recorre hasta n intervalos. La inserción seguida de ordenación tiene una cota superior $O(n \log n)$; cancelar requiere $O(n)$. La disponibilidad de una sala recorre sus n reservas. El almacenamiento crece con la cantidad total de reservas.

La solución está pensada para una ejecución local y un mismo día. Los datos no sobreviven al cierre del proceso; no existe control de concurrencia ni de identidad de usuarios. Estas funciones podrían añadirse si cambiara el alcance del ejercicio.

7. Prompt reutilizable sugerido

El siguiente texto es una propuesta para reproducir la actividad; no se presenta como un registro de prompts previos.

«Genera una clase BookingCalendar en Python con book, cancel y get_available_slots compatible con cot_test.py. Usa intervalos de un día, evita traslapes dentro de una sala, permite horarios consecutivos y muestra disponibilidad en tabla HH:MM. Entrega un resumen de diseño, diagrama de clases, código PEP 8, demostración y pruebas para entradas inválidas. Conserva intacto el archivo del docente.»

Liga del proyecto

[Consultar código, pruebas y evidencia en GitHub](#)